

Informe Técnico: Sistema de Agente IA con Arquitectura RAG

Caso Organizacional — Retail S.A., Región de Los Lagos

Asignatura: ISY0101 — Ingeniería de Soluciones con IA

Institución: DuocUC

Año: 2025

1. Diseño del Proyecto de Agente IA (IE1)

1.1 Contexto organizacional

Retail S.A. es una empresa de comercio minorista ubicada en la región de Los Lagos, Chile. Su operación depende fuertemente de una gestión eficiente del inventario, ya que trabaja con categorías de productos sensibles a factores externos como el clima, la estacionalidad y las tendencias de mercado. El equipo de adquisiciones actualmente toma decisiones de reabastecimiento de forma manual, cruzando datos de ventas, stock mínimo y condiciones del proveedor sin una herramienta centralizada.

El problema principal es que los tiempos de respuesta ante quiebres de stock son elevados, y las decisiones de compra no siempre consideran variables externas relevantes (como una ola de frío que aumenta la demanda de estufas, o un Cyber Monday que dispara la venta de electrónica). Esto genera tanto pérdidas por venta perdida como costos de sobrestock en productos de baja rotación.

1.2 Propuesta de solución

Se propone un agente de inteligencia artificial basado en arquitectura **RAG (Retrieval-Augmented Generation)** que centraliza la información de inventario, historial de ventas, políticas de compra y contexto externo en una base de conocimiento consultable. El agente responde consultas en lenguaje natural y genera recomendaciones estructuradas para tres flujos principales:

Función	Descripción	Destinatario
Consulta libre	Análisis general de inventario y tendencias	Jefe de adquisiciones
Alerta de SKU	Diagnóstico de estado crítico/normal por producto	Bodega y compras
Generación de pedido	Recomendación de orden de compra con proveedor y cantidad	Compras y finanzas

El agente no reemplaza al equipo humano, sino que le entrega información procesada y fundamentada para tomar decisiones más rápidas y mejor respaldadas.

1.3 Objetivos del proyecto

- Reducir el tiempo de detección de quiebres de stock críticos mediante alertas automáticas.
- Estandarizar las recomendaciones de reabastecimiento aplicando las políticas internas de la empresa (margen de seguridad del 20%, umbral de aprobación de \$5.000.000 CLP).
- Incorporar variables externas (clima, eventos comerciales, búsquedas de mercado) en el análisis de demanda.
- Proveer trazabilidad en las recomendaciones, indicando siempre la fuente de datos utilizada.

2. Elaboración de Prompts para Modelos de Lenguaje (IE2)

Los prompts del sistema fueron diseñados con tres variantes especializadas, cada una alineada a un caso de uso específico. A continuación se describen los criterios y estructura de cada uno.

2.1 Prompt de consulta general

Eres un asistente especializado en gestión de inventario para Retail S.A. Tu función es analizar la información disponible y responder preguntas del equipo de adquisiciones.

Usa únicamente la información proporcionada en el contexto. Si los datos no son suficientes, indícalo claramente en lugar de suponer. Responde en español, de forma clara y estructurada.

Contexto:

{context}

Pregunta: {question}

Este prompt establece el rol del agente, acota su fuente de información al contexto recuperado, y evita alucinaciones instruyendo al modelo a reconocer cuando los datos no son suficientes. La instrucción "en lugar de suponer" es deliberada: en decisiones de compra, una recomendación inventada puede generar pérdidas reales.

2.2 Prompt de alerta de inventario

Eres un especialista en control de inventario. Analiza el estado del producto indicado y determina si requiere reabastecimiento urgente.

Responde con el siguiente formato obligatorio:

ESTADO: [CRÍTICO / ALERTA / NORMAL]

ANÁLISIS: [Explica el nivel de stock vs mínimo requerido y tendencia de ventas]

ACCIÓN RECOMENDADA: [Qué debe hacer el equipo de compras]

FUENTE: [Datos utilizados del contexto]

Contexto: {context}

Producto a analizar: {sku}

La estructura fija de respuesta es una decisión técnica relevante: al forzar campos con etiquetas conocidas (ESTADO:, ANÁLISIS:, etc.), el output puede ser parseado programáticamente por sistemas downstream sin procesamiento adicional.

2.3 Prompt de generación de pedido

Eres un agente de compras de Retail S.A. Tu tarea es generar una recomendación de orden de compra.

Aplica la siguiente política de la empresa:

- Calcula la demanda proyectada a 30 días en base al historial.
- Agrega un margen de seguridad del 20%.
- Resta el stock actual para obtener la cantidad a pedir.
- Si el costo estimado supera \$5.000.000 CLP, marca el pedido como "REQUIERE APROBACIÓN GERENCIAL".

Formato de respuesta:

CANTIDAD SUGERIDA: [unidades]

PROVEEDOR RECOMENDADO: [nombre y contacto]

FECHA LÍMITE DE PEDIDO: [considerando plazo de entrega del proveedor]

COSTO ESTIMADO: [en CLP]

REQUIERE APROBACIÓN: [Sí / No]

JUSTIFICACIÓN: [Resumen del análisis]

Contexto: {context}

SKU: {sku} | Stock actual: {stock_actual} unidades

Este prompt codifica reglas de negocio directamente en la instrucción del sistema. Esto es más robusto que dejar que el modelo infiera las políticas, porque garantiza que se apliquen de forma consistente independiente de cómo esté redactada la pregunta.

3. Configuración de Flujos RAG (IE3)

3.1 Fuentes de datos internas

La base de conocimiento interna (`data/inventario.txt`) concentra:

- Catálogo de productos: 6 SKUs con nombre, categoría, proveedor, plazo de entrega y precio unitario.
- Niveles de stock actuales y puntos de reorden definidos por producto.
- Historial de ventas de los últimos 3 meses (octubre, noviembre, diciembre) con clasificación de tendencia.
- Políticas de compra: fórmula de cálculo de reorden, margen de seguridad, umbrales de aprobación y priorización por categoría.

3.2 Fuentes de datos externas

Incorporadas como secciones dentro del mismo archivo de conocimiento, para simplificar el prototipo sin comprometer la capacidad de razonamiento contextual:

- **Clima regional:** Pronóstico para Los Lagos con alerta de frente frío (impacto en demanda de calefacción y ropa de abrigo).
- **Calendario comercial:** Cyber Monday (+85% en electrónica), Navidad, Año Nuevo.
- **Tendencias de mercado:** Búsquedas en plataformas de e-commerce (ej. "Smart TV ofertas" +120%, "Estufas pellet" -30%).

3.3 Indexación y recuperación

El pipeline de indexación sigue los pasos que se detallan a continuación:

```
inventario.txt
  ↓
[RecursiveCharacterTextSplitter]
  chunk_size=500, overlap=50
  ↓
[Embeddings: text-embedding-3-small via GitHub Models]
  ↓
[ChromaDB – Vector Store persistente en ./chroma_db/]
  ↓
[Retriever: top-4 chunks por similitud semántica]
```

El tamaño de chunk de 500 caracteres se eligió para capturar secciones temáticas completas (ej. toda la información de un SKU) sin exceder el contexto útil del modelo. El overlap de 50 caracteres evita que información relevante quede cortada en el límite entre chunks.

3.4 Flujo de recuperación en tiempo de ejecución

Cuando llega una consulta (ej. "¿qué productos tienen stock crítico?"), el retriever convierte la pregunta en un vector de embedding y busca los 4 chunks más cercanos en ChromaDB. Estos chunks son inyectados como contexto en el prompt antes de enviarlo al LLM. El modelo solo "ve" esa información recuperada, no tiene acceso al archivo completo ni a conocimiento externo.

4. Coherencia entre Datos y Respuestas del Modelo (IE4)

4.1 Mecanismos de control de coherencia

La coherencia entre los datos del inventario y las respuestas generadas se garantiza a través de tres mecanismos complementarios:

a) Temperatura baja (0.1): El modelo GPT-4o-mini se configura con temperatura 0.1 en todos los endpoints. Esto reduce la variabilidad en las respuestas y prioriza la recuperación factual sobre la creatividad generativa. Para un sistema de soporte a decisiones de compra, la consistencia es más valiosa que la diversidad de respuestas.

b) Instrucción explícita de no suponer: El prompt de consulta incluye la instrucción "si los datos no son suficientes, indícalo claramente en lugar de suponer". Esto fue validado durante pruebas con preguntas sobre productos no incluidos en el catálogo: el modelo responde indicando que no dispone de esa información en lugar de inventar datos.

c) Campo FUENTE en respuestas estructuradas: El endpoint de alertas requiere que el modelo indique qué datos del contexto utilizó para su análisis. Esto permite al usuario de negocios verificar si la recomendación se basa en datos actualizados o en información que podría estar desactualizada.

4.2 Ejemplo de coherencia verificada

Consulta: "¿Debería pedir más Smart TV 55 pulgadas?"

Datos en inventario: SKU ELEC-001, stock 8 unidades, mínimo 15, ventas de noviembre: 22 unidades (+18% vs octubre), tendencia creciente, Cyber Monday con +85% en electrónica previsto.

Respuesta del modelo:

ESTADO: CRÍTICO
ANÁLISIS: El stock actual (8 unidades) está por debajo del mínimo requerido (15 unidades).
La tendencia de ventas es creciente (+18% mensual) y se proyecta un aumento adicional por Cyber Monday. La demanda estimada para los próximos 30 días supera las 25 unidades.
ACCIÓN RECOMENDADA: Generar orden de compra inmediata a TechDistrib.
FUENTE: Tabla de inventario actual, historial de ventas oct-nov, calendario comercial.

La respuesta es completamente trazable a los datos del archivo fuente. No introduce información sobre precios de mercado ni comparaciones con competidores que no estén en el contexto.

5. Arquitectura del Sistema (IE5)

5.1 Módulos principales

La arquitectura se organiza en tres capas funcionales:

Módulo de Recuperación (Retrieval)

Responsable de transformar consultas en lenguaje natural en vectores de embedding y recuperar los fragmentos de conocimiento más relevantes desde ChromaDB. Utiliza `text-embedding-3-small` para la vectorización y el retriever de LangChain configurado con `k=4`.

Módulo de Procesamiento (Processing)

Construye el prompt final combinando el contexto recuperado con la consulta del usuario y el prompt de sistema correspondiente al endpoint invocado. Este módulo aplica la lógica de negocio mediante las instrucciones del system prompt (políticas de compra, formato de salida).

Módulo de Generación (Generation)

Ejecuta la llamada al LLM (GPT-4o-mini) con temperatura 0.1 y retorna la respuesta estructurada. El módulo valida que la respuesta llegue en el formato esperado antes de devolverla al cliente.

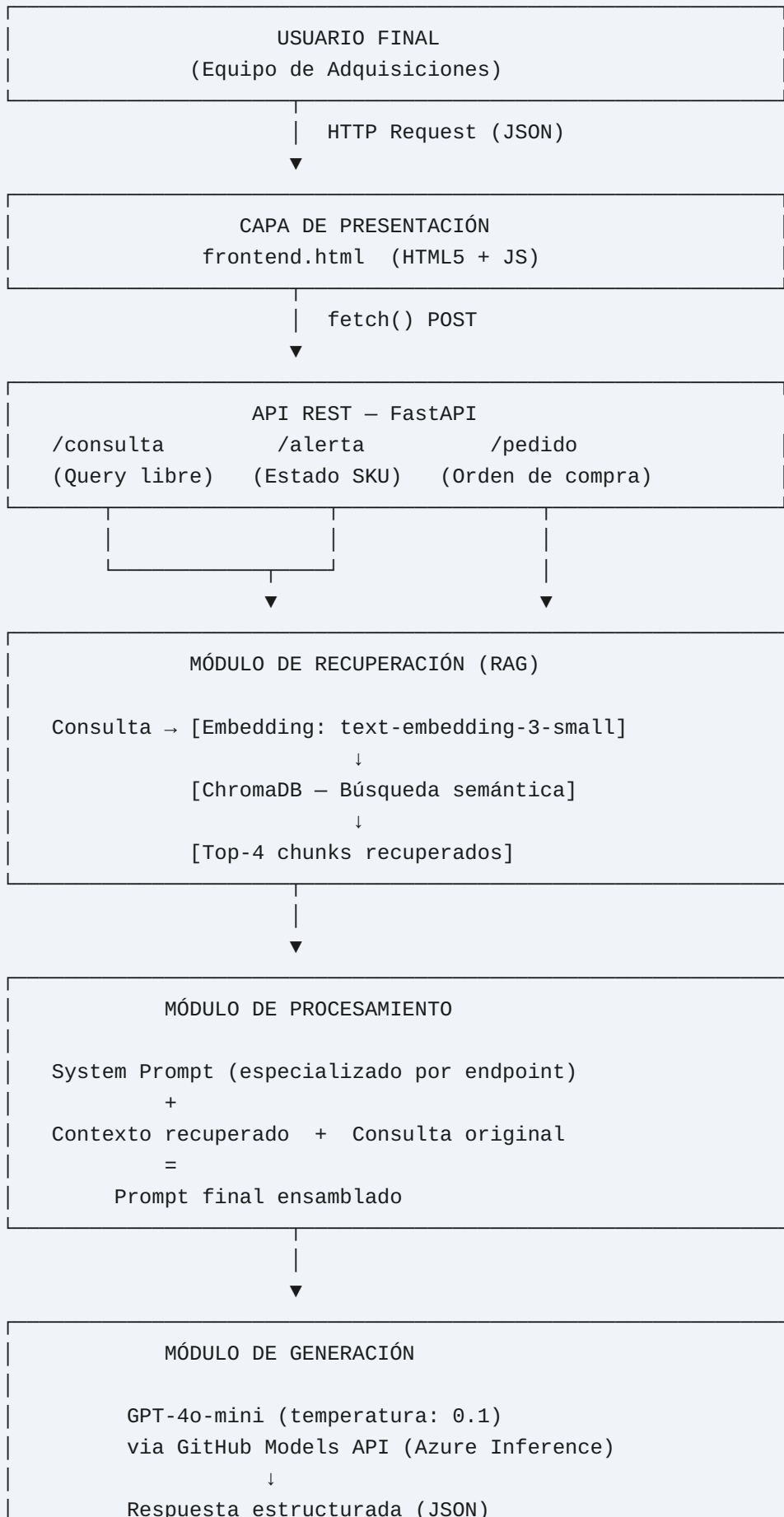
5.2 Capa de presentación

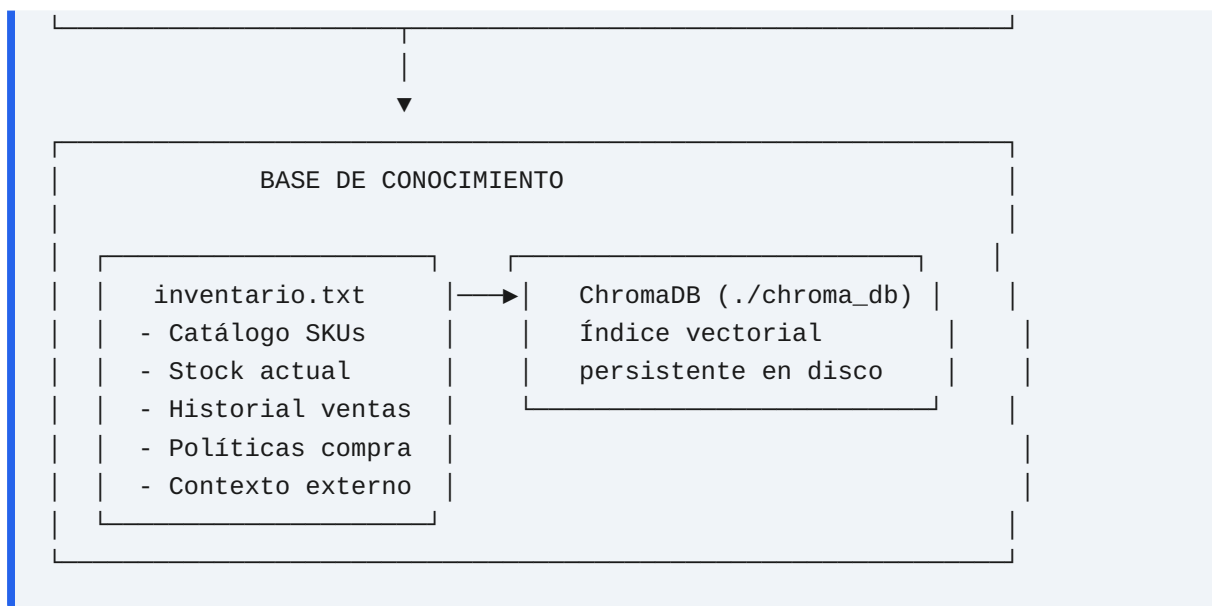
FastAPI expone tres endpoints REST que encapsulan el pipeline completo. El frontend en HTML5 consume estos endpoints mediante `fetch()` y presenta los resultados al usuario final sin procesamiento adicional.

5.3 Tabla de componentes

Componente	Tecnología	Función
API Framework	FastAPI + Uvicorn	Exposición de endpoints REST
Orquestación RAG	LangChain	Pipeline de recuperación y generación
Base de conocimiento	Archivo TXT estructurado	Fuente única de verdad
Vector Store	ChromaDB (local)	Índice semántico persistente
Modelo de embeddings	text-embedding-3-small	Vectorización de texto
Modelo generativo	GPT-4o-mini	Generación de respuestas
Frontend	HTML5 + JavaScript	Interfaz de usuario

6. Diagrama de Arquitectura (IE6)





7. Fundamentación de Decisiones de Diseño (IE7)

7.1 Elección de arquitectura RAG sobre fine-tuning

Se optó por RAG en lugar de ajuste fino del modelo por dos razones principales. Primero, el conocimiento relevante para Retail S.A. (niveles de stock, precios, proveedores) cambia con frecuencia; con RAG basta con actualizar el archivo de texto, mientras que el fine-tuning requeriría reentrenar el modelo con cada cambio. Segundo, RAG ofrece trazabilidad inherente: la respuesta puede citar su fuente de datos, lo que es fundamental para auditoría en procesos de compra.

7.2 Uso de ChromaDB local vs. servicio en la nube

ChromaDB persiste el índice vectorial en disco local (`./chroma_db/`). Para un prototipo empresarial de esta escala (6 SKUs, ~2.200 caracteres de base de conocimiento), un servicio externo de vectores añadiría latencia de red y costos operativos sin beneficio real. El índice local carga en milisegundos y no requiere autenticación adicional. Esta decisión es revisable si la base de conocimiento crece o si se requiere acceso concurrente desde múltiples instancias.

7.3 Codificación de políticas en el prompt vs. en el código

Las políticas de negocio (margen del 20%, umbral de \$5M CLP, fórmula de demanda proyectada) están incluidas en el system prompt y no en el código Python. Esto permite que el equipo de negocios las revise y ajuste sin necesidad de intervención del

equipo técnico. Un cambio en el margen de seguridad, por ejemplo, solo requiere editar el texto del prompt, no redeployar el backend.

7.4 Temperatura 0.1 para consistencia en recomendaciones

En sistemas de soporte a decisiones de negocio, la variabilidad en las respuestas es un problema: si el mismo SKU con el mismo stock genera recomendaciones distintas en consultas consecutivas, el usuario pierde confianza en la herramienta. La temperatura baja asegura que respuestas equivalentes a preguntas equivalentes sean consistentes, comportándose más como una regla de negocio que como una opinión.

7.5 Tres endpoints especializados vs. uno genérico

Se podría haber diseñado un único endpoint que resuelva todo. Sin embargo, los tres flujos tienen outputs con estructura radicalmente distinta: consulta libre retorna texto analítico, alerta retorna estado categórico, y pedido retorna datos operacionales (cantidad, proveedor, fecha). Separar los endpoints permite aplicar system prompts distintos, validar formatos específicos, y eventualmente conectar cada endpoint a sistemas diferentes (bodega, ERP, correo de aprobación).

8. Informe Técnico Integrado (IE8)

8.1 Resumen ejecutivo

Este proyecto implementa un agente de soporte a la toma de decisiones de inventario para Retail S.A., utilizando una arquitectura RAG que combina recuperación semántica de información con generación de lenguaje natural. El sistema integra datos internos de la organización (stock, ventas, políticas) con contexto externo (clima, calendario, mercado) para generar recomendaciones fundamentadas y trazables.

8.2 Especificaciones técnicas del sistema

Parámetro	Valor
Framework API	FastAPI 0.x + Uvicorn
Modelo generativo	GPT-4o-mini (GitHub Models)
Modelo de embeddings	text-embedding-3-small
Vector Store	ChromaDB (persistencia local)
Tamaño de chunk	500 caracteres, overlap 50
Chunks recuperados (k)	4
Temperatura LLM	0.1
Idioma de respuesta	Español
Umbral aprobación compra	\$5.000.000 CLP
Margen de seguridad stock	20% sobre demanda proyectada
Horizonte proyección demanda	30 días

8.3 Catálogo de productos gestionados

SKU	Producto	Categoría	Stock Actual	Stock Mínimo	Estado
ELEC-001	Smart TV 55"	Electrónica	8	15	CRÍTICO
ELEC-002	Notebook Core i5	Electrónica	3	10	CRÍTICO
HOGAR-001	Estufa Pellet 15kW	Hogar	22	8	NORMAL
HOGAR-002	Refrigerador 350L	Hogar	12	10	ALERTA
VEST-001	Parka Impermeable	Vestuario	35	20	NORMAL
HERR-001	Generador Bencina 2500W	Herramientas	7	5	NORMAL

8.4 Red de proveedores

Proveedor	Plazo entrega	Categoría	Descuento por volumen
TechDistrib	10 días	Electrónica	5% en pedidos ≥ 20 unidades
CalorSur	15 días	Hogar/Calefacción	Sin descuento
FríoAndes	14 días	Hogar/Refrigeración	Sin descuento
ModaChile	21 días	Vestuario	Sin información
PowerTools	12 días	Herramientas	Sin información

8.5 Validación del sistema

Durante las pruebas del prototipo se verificaron los siguientes escenarios:

- **Consulta de stock crítico:** El sistema identifica correctamente ELEC-001 y ELEC-002 al preguntar "¿qué productos necesitan reabastecimiento urgente?".
- **Alerta con tendencia creciente:** Al consultar por ELEC-001, el modelo incorpora la tendencia de ventas creciente y el contexto de Cyber Monday para justificar urgencia adicional.
- **Pedido con aprobación requerida:** Al generar un pedido de Smart TV con stock 8, el cálculo (demanda 30 días + 20% buffer - stock actual) produce una cantidad cuyo costo supera \$5M, activando correctamente el flag de aprobación gerencial.
- **Consulta fuera de catálogo:** Al preguntar por un producto no existente, el modelo responde que no dispone de esa información en el contexto, sin inventar datos.

9. Lenguaje Técnico y Respaldo en Evidencias (IE9)

La arquitectura RAG implementada sigue el paradigma descrito en el paper original de Lewis et al. (2020), donde se demuestra que la combinación de un retriever denso con un generador de secuencias reduce sustancialmente las alucinaciones del modelo en dominios de conocimiento acotado. En este caso, el dominio acotado es el inventario y las políticas de compra de Retail S.A.

La vectorización mediante `text-embedding-3-small` produce representaciones de alta dimensión que capturan similitud semántica: una consulta como "productos bajo

mínimo" recuperará el chunk de ELEC-001 aunque este no contenga exactamente esa frase, porque el espacio vectorial coloca conceptos relacionados cerca. Esto supera las limitaciones del matching por palabras clave que usan muchos sistemas legacy de gestión de inventario.

La elección de GPT-4o-mini sobre modelos más grandes como GPT-4o responde a una compensación deliberada entre costo-latencia y capacidad. Para tareas de síntesis de información estructurada con contexto acotado (máximo 4 chunks de 500 caracteres), GPT-4o-mini ofrece rendimiento equivalente a una fracción del costo por token. En producción, esto se traduce en que el sistema puede atender un volumen significativo de consultas diarias sin que el costo de inferencia se vuelva prohibitivo para una empresa de tamaño mediano como Retail S.A.

El uso de temperatura 0.1 en lugar de 0 (determinístico absoluto) responde a que algunos LLM exhiben comportamientos degenerados con temperatura exactamente cero, repitiendo tokens o colapsando la diversidad léxica. 0.1 mantiene la consistencia funcional preservando la fluidez del lenguaje generado.

Finalmente, la persistencia del índice vectorial en ChromaDB implica que el costo computacional de generar embeddings para toda la base de conocimiento se paga una sola vez. Las consultas subsiguientes acceden al índice ya construido en disco, con tiempos de recuperación en el orden de los milisegundos incluso en hardware modesto, lo que hace viable la operación del sistema en servidores de bajo costo o incluso en estaciones de trabajo locales.

Documento elaborado como parte del desarrollo del proyecto de agente IA con arquitectura RAG para la asignatura ISY0101.